

Introduction in PL/SQL – Procedural Language extension of SQL

1. General characteristics:

- PL/SQL constructs contain procedural control structures and SQL descriptive commands;
- PL/SQL is a block-structured procedural language, allowing programs to be divided into logical blocks;
- PL/SQL blocks are processed by the PL/SQL engine, which can be run on the ORACLE SERVER or a development tool (e.g., APEX, Oracle Forms, Reports, JDeveloper, etc.);
- Many ORACLE tools have their own PL/SQL engine (e.g., APEX, Oracle Forms, Reports, JDeveloper, etc.);
- SQL data types can be used in PL/SQL;
- Programming in PL/SQL is modularized – using blocks that group instructions.

2. PL/SQL blocks:

Any PL/SQL unit contains one or more blocks, either completely separate or nested.

Components of a PL/SQL block:

A PL/SQL block consists of up to 3 sections: **declarative (optional)**, **executable (mandatory)**, and **exception handling (optional)**.

DECLARE (*Optional*)

variables cursors, exceptions

BEGIN (*Mandatory*)

SQL commands (ensures database access),

PL/SQL procedural programming structures

EXCEPTION (*Optional*)

actions executed when an error occurs

END; (*Mandatory*)

Notes:

- ✓ SQL commands ensure database access.
- ✓ Operations performed with PL/SQL variables within procedural instructions do not require database access.
- ✓ Use (;) after each SQL or PL/SQL control instruction.
- ✓ The PL/SQL block ends with (;).
- ✓ Use (/) to launch an anonymous block in the SQL buffer.
- ✓ An error in PL/SQL is treated as an exception.

Types of PL/SQL Blocks:

- Anonymous blocks
- Stored functions and application functions
- Stored procedures and application procedures
- Packages
- Triggers on the database/application

Anonymous Blocks:

- Do not have a name.
- Are not stored in the database.
- Declared inline, where execution is desired.
- Executed at runtime.

Example:

```
DECLARE
    v_variable varchar2(5);
BEGIN
    SELECT column INTO v_variable FROM table;
EXCEPTION
    WHEN exception THEN action
END;
/
```

Nested Anonymous Blocks:

- Multiple blocks can be nested.
- These can be labeled with `<<block_label>>`, and variables within the blocks can be used as: `block_label.variable`.

```
BEGIN
.....
    <<block_label>>
    DECLARE
    .....
    BEGIN
    .....
    END block_label;
END;
/
```

Procedures, Functions:

- PL/SQL blocks with a name.
- Can be stored at the ORACLE SERVER level (stored procedures/functions) or at the application level (APEX or Forms & Reports).

Examples:

```
CREATE [OR REPLACE] PROCEDURE procedure_name
IS
.....
BEGIN
.....
[EXCEPTION]
.....
END;
```

```
CREATE [OR REPLACE] FUNCTION function_name
RETURN data_type
IS
BEGIN
.....
RETURN value
[EXCEPTION]
.....
END;
```

Program Packages: Group procedures, functions.

Database Triggers: PL/SQL blocks associated with tables (base or virtual) and automatically executed when a manipulation command occurs.

Application Triggers: PL/SQL blocks associated with application events (e.g., mouse movement, button press) and automatically executed.

3. Operators in PL/SQL

Operator	Characteristics
+, -, *, /, ** (op. exponențial)	Arithmetic operators
AND, OR, NOT	Logical operators
<, >, =, >=, <=, <>, !=	Comparison operators
BETWEEN ... AND ...	Range check operator
IN(listă)	List check operator
LIKE	Pattern matching operator % - as many characters; _ - one character;
IS NULL	Checks if a variable is NULL
	Concatenation operator
@	Remote connection operator
& sau &&	Substitution variable addressing operators
:=	Assignment operator

4. SQL functions supported in PL/SQL

- All types of SQL functions (including group functions in the SELECT statement) are supported in descriptive instructions.
- PL/SQL instructions:
 - Supports single-row functions: numeric, character, date, conversion, etc.
 - Does **NOT** support group functions (*SUM, MIN, MAX, AVG, COUNT, STDDEV*) or the *DECODE* function. For example, constructions like:
IF *DECODE(...)* THEN ... or IF *AVG(...)* THEN ... or *a:=max(b,c)* cannot be used.
To find the maximum of 2 or more values, the single-row function **GREATEST** (LEAST for minimum) can be used.

5. Conversions in PL/SQL Blocks:

- PL/SQL dynamically converts data types (e.g., a numeric value to a char variable).
- Implicit conversions: character <-> numeric and character <-> date.
- Explicit conversions: use *TO_DATE, TO_NUMBER, TO_CHAR* functions.

6. SQL Commands Supported in PL/SQL

- PL/SQL allows the use of data manipulation commands (DML):
 - ✓ SELECT
 - ✓ INSERT
 - ✓ UPDATE
 - ✓ DELETE

- PL/SQL allows the use of transaction control commands:
 - ✓ COMMIT
 - ✓ ROLLBACK
 - ✓ SAVEPOINT

Note: A PL/SQL block is not a transaction. Commit/Rollback/Savepoint commands are independent of the block but can appear within it.

- PL/SQL does **NOT** support data definition commands (DDL):
 - ✓ CREATE
 - ✓ ALTER
 - ✓ DROP
 - ✓ RENAME
 - ✓ TRUNCATE
- PL/SQL does **NOT** support data control language commands (DCL):
 - ✓ GRANT
 - ✓ REVOKE

Note: DDL and DCL commands can be used through the EXECUTE IMMEDIATE command or the DBMS_SQL package.

7. Displaying the results with PL/SQL

For properly displaying the results, similar to SQL, we need to use the following functions:

DBMS_OUTPUT.PUT_LINE (...); -- displays the result and moves to the next line

DBMS_OUTPUT.PUT(...); -- displays the message on the current line

They have only one parameter, so if you want to display multiple variables you have to concatenate the result.

SET SERVEROUTPUT ON – setting the display on the screen